

Куча. Сортировка кучей.

Рекурсия

Определение рекурсии

Рекурсия в программировании – вызов функции из неё же самой.

Пример рекурсивной функции вычисления факториала
Реализована формула $n! = n * (n-1)!$

```
long fact(int n) {  
    return (n<=1) ? 1 : n * fact(n-1);  
}  
void main() {  
    ...  
    cout << n<<"! = " << fact(n)<<endl;  
    ...  
}
```

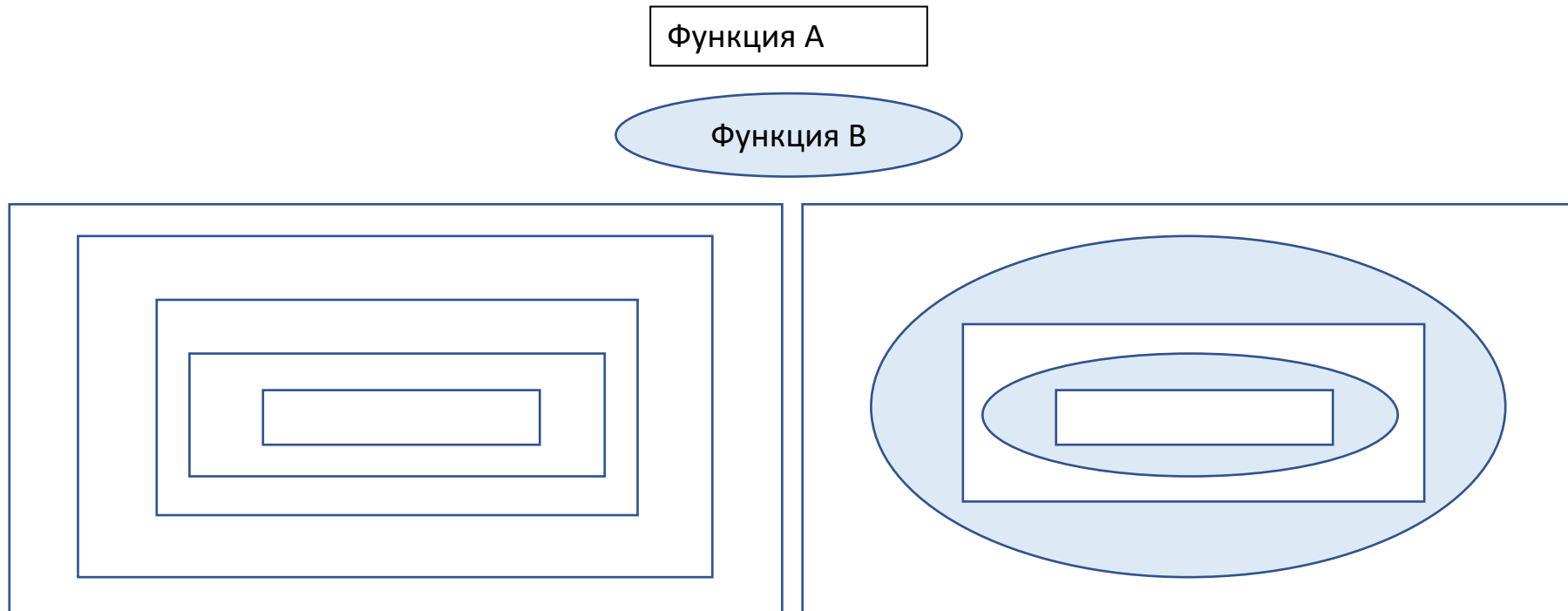
Задача сводится «сама к себе» с другими параметрами

Виды рекурсии

Различают прямую и косвенную рекурсии.

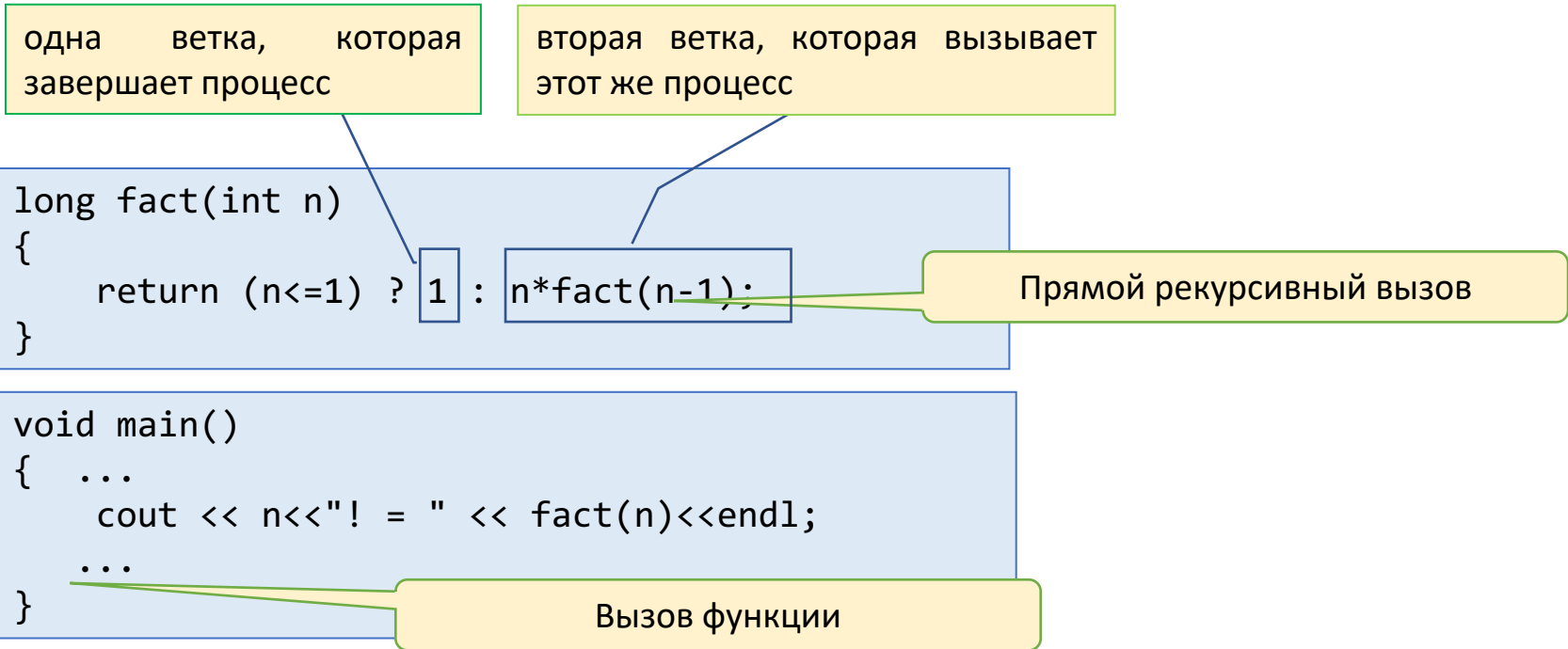
Функция называется **прямо рекурсивной**, если содержит в своем теле вызов самой себя (с измененными параметрами!).

Если же функция1 вызывает функцию2, которая в свою очередь вызывает функцию1, то функция1 называется **косвенно рекурсивной**.



Прямая рекурсия

В рекурсивных алгоритмах всегда есть



Рекурсивная функция без «дна»

Вызов функции

```
void main() { ...  
    cout << n<<"! = " <<fact(5)<<endl;  
    ... }
```

Рекурсивная функция вычисления факториала

```
long fact(int n) {  
    return      n*fact(n-1);  
}
```

Рекурсивное
обращение к
функции

Вглубь рекурсии



fact(5) = 5 * fact(4)

fact(4) = 4 * fact(3)

fact(3) = 3 * fact(2)

fact(2) = 2 * fact(1)

fact(1) = 1 * fact(0)

fact(0) = 0 * fact(-1)

fact(-1) = -1 * fact(-2)

fact(-2) = -2 * fact(-3)

Рекурсивная функция

Вызов функции

```
void main()
{
    ...
    cout << n << "! = " << fact(5) << endl;
    ...
}
```

Рекурсивная функция вычисления факториала

```
long fact(int n)
{
    return (n <= 1) ? 1 : n * fact(n - 1);
}
```

Условие выхода из рекурсии

Обязательно предусмотрите останов рекурсии!



fact(5) = 120

fact(5) = 5 * fact(4) fact(5) = 24 * 5

fact(4) = 4 * fact(3) fact(4) = 6 * 4

fact(3) = 3 * fact(2) fact(3) = 3 * 2

fact(2) = 2 * fact(1) fact(2) = 2 * 1

fact(1) = 1 * ~~fact(0)~~ fact(1) = 1

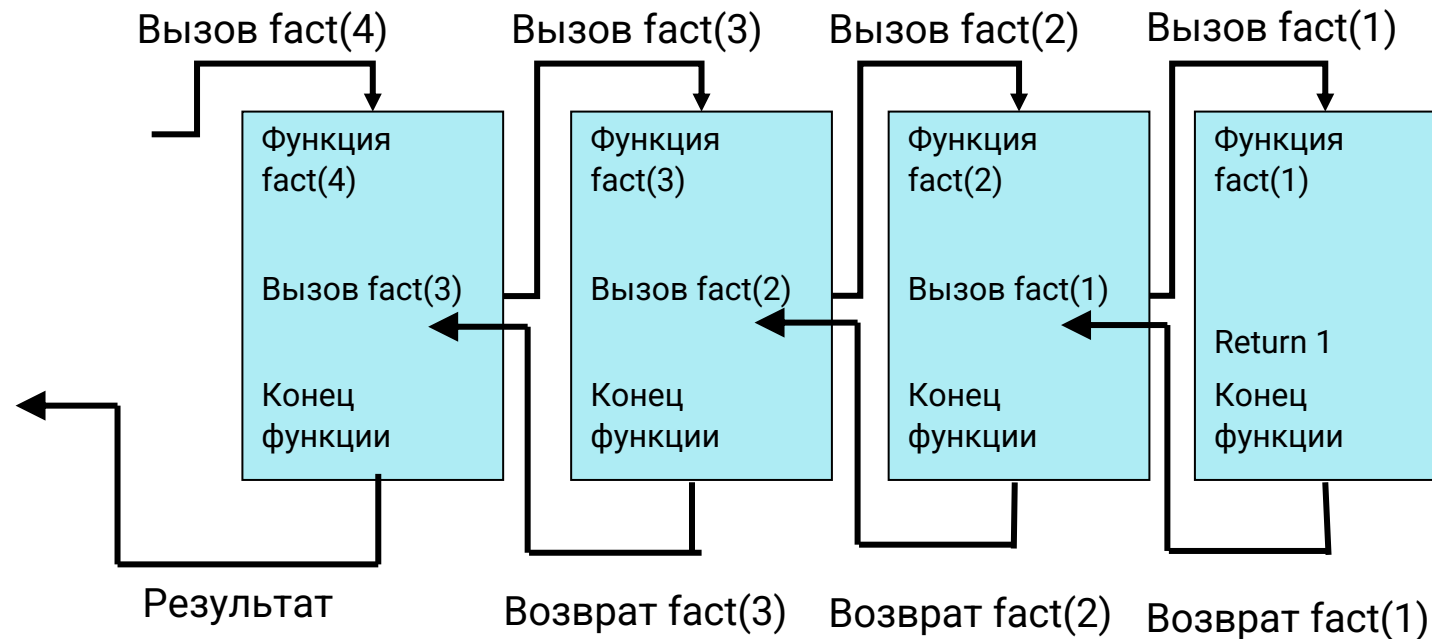
fact(1) = 1

Останов рекурсии

Вглубь рекурсии

Возврат рекурсии

Рекурсия. Как это работает (модель)



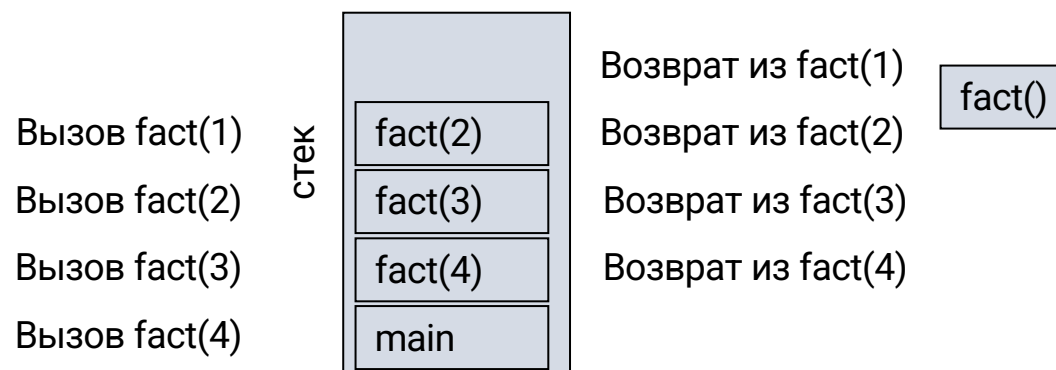
Количество вызовов подпрограммы самой себя называется **глубиной рекурсии** или **глубиной рекурсивных вызовов**.

Стек вызовов

Стек вызовов – LIFO-стек, хранящий информацию для возврата управления из выполнившейся функции к вызвавшей её.

При каждом вызове любой подпрограммы (процедуры или функции), в стек вызовов заносятся «адрес возврата» (команда вызвавшей процедуры, с которой выполнение продолжится после завершения подпрограммы), значения локальных переменных, аргументы и другая информация.

Рассмотрим работу программы вычисления факториала для $n = 4$.



Стек вызовов

При вызове функции

1) в стек помещаются

- адрес возврата,
- содержимое регистров процессора,
- состояние локальных ячеек (переменных) вызывающей функции (ссылка на ее запись активации),
- возвращаемое значение (его адрес) и
- передаваемые функции фактические параметры (аргументы)

2) выполняется переход по адресу на вызываемую функцию,

Стек вызовов

Вызываемая функция

- извлекает переданные фактические параметры,
- выполняет вычисления,
- помещает результат по указанному в стеке адресу.

При завершении работы вызываемая функция

- восстанавливает состояние регистров,
- восстанавливает локальные ячейки вызывающей функции,
- выталкивает из стека адрес возврата для передачи управления по этому адресу.

Рекурсия

Плюсы рекурсии:

- Для составления функции используется постановка задачи, поэтому ряд задач легче запрограммировать в виде рекурсии

Минусы рекурсии:

- большие расходы памяти,
- медленная работа при большой глубине рекурсии.

Рекурсия vs итерация

При итерации может произойти заикливание.

При использовании рекурсии - переполнение памяти (стека)

При использовании рекурсивных функций и разворачивании рекурсивных вызовов, на каждый конкретный вход в функцию будет израсходована память стека под все локальные переменные, фактические параметры, адрес возврата и пр.

Использование локальных переменных должно быть сведено к минимуму, иначе память (стека) может закончиться раньше, чем будет получен результат.

```
long fib(long n) {  
    long a = 1;  
    long b = 1;  
    long c;  
    for (int i = 2; i <= n; i++)  
        {c = a+b; a = b; b = c;}  
    return c;  
}
```

```
long fibrec(long n) {  
    if (n < 2)  
        {return 1;}  
    else  
        {return fibrec(n-1) + fibrec(n-2);}  
}
```

Пример рекурсии

Задана функция

пользователя

```
double F (double x)
{
    if (abs(x - 2) < 2)
        return 8;
    else
        {double F1=0.5*pow((x-F(x-2)),2);
        return F1;}
}
```

Значение F (8.0)
равно

Ответ: 18



Прямой ход (вглубь рекурсии)

$$F(8) = 0.5 * (8 - F(8 - 2))^2;$$

$$F(6) = 0.5 * (6 - F(6 - 2))^2;$$

$$F(4) = 0.5 * (4 - F(4 - 2))^2;$$

$$F(2) = 8$$

Останов рекурсии

Обратный ход (возврат рекурсии)

$$F(4) = 0.5 * (4 - 8)^2 = 8$$

$$F(6) = 0.5 * (6 - 8)^2 = 2$$

$$F(8) = 0.5 * (8 - 2)^2 = 18$$

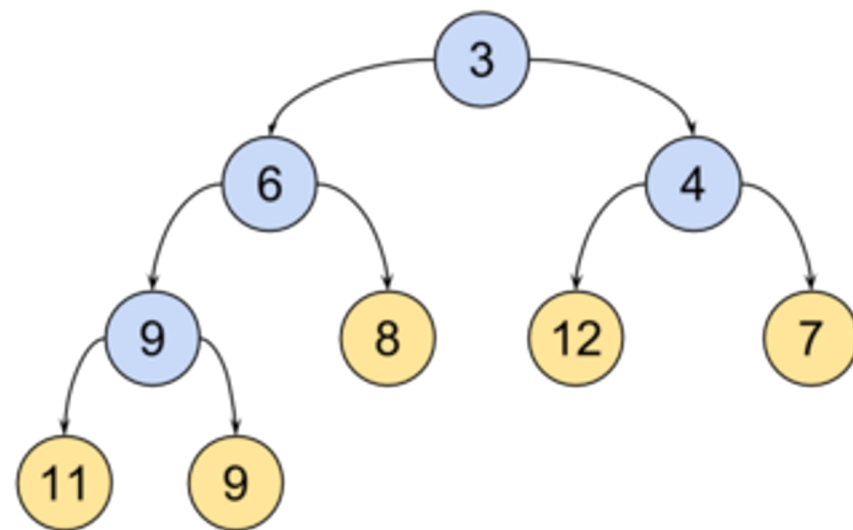
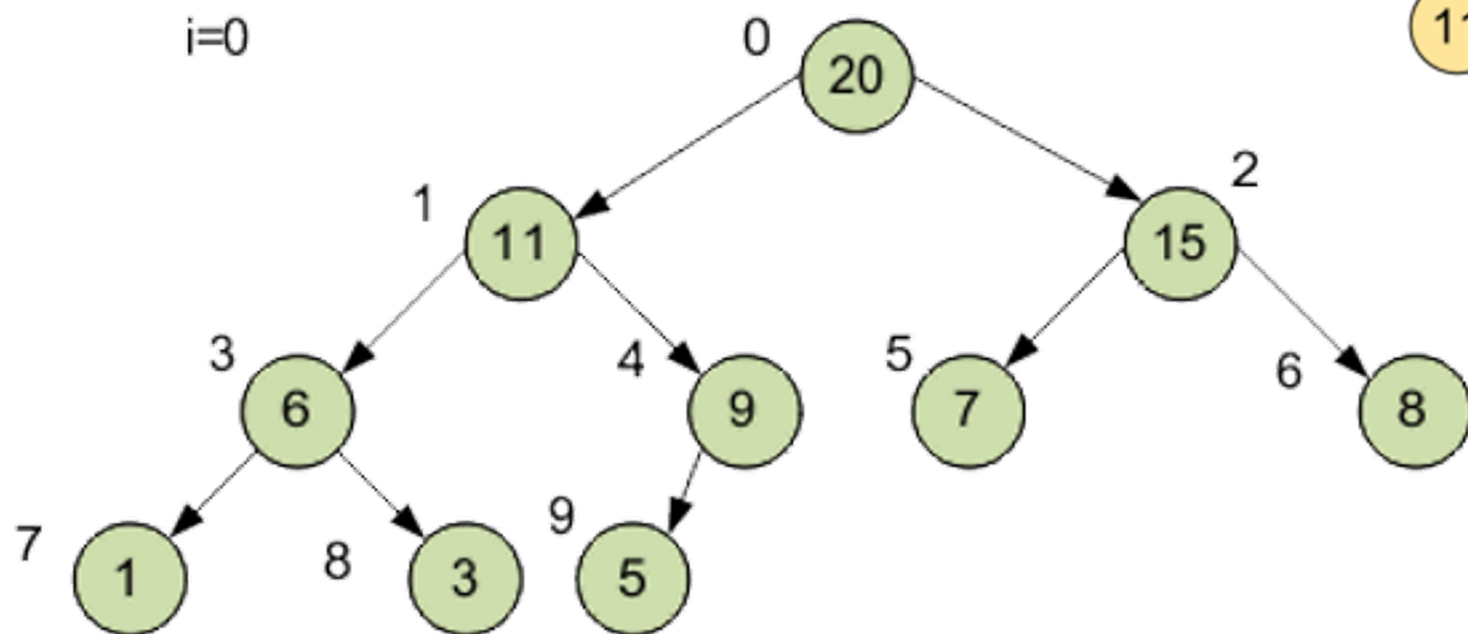
Сортировка кучей

Двоичная куча

- Двоичная куча представляет собой полное бинарное дерево, для которого выполняется основное свойство кучи: приоритет каждой вершины больше приоритетов её потомков.
- В простейшем случае приоритет каждой вершины можно считать равным её значению. В таком случае структура называется ***max-куча***, поскольку корень поддерева является максимумом из значений элементов поддерева.
- В качестве альтернативы, если сравнение перевернуть, то наименьший элемент будет всегда корневым узлом, такие кучи называют ***min-кучами***.

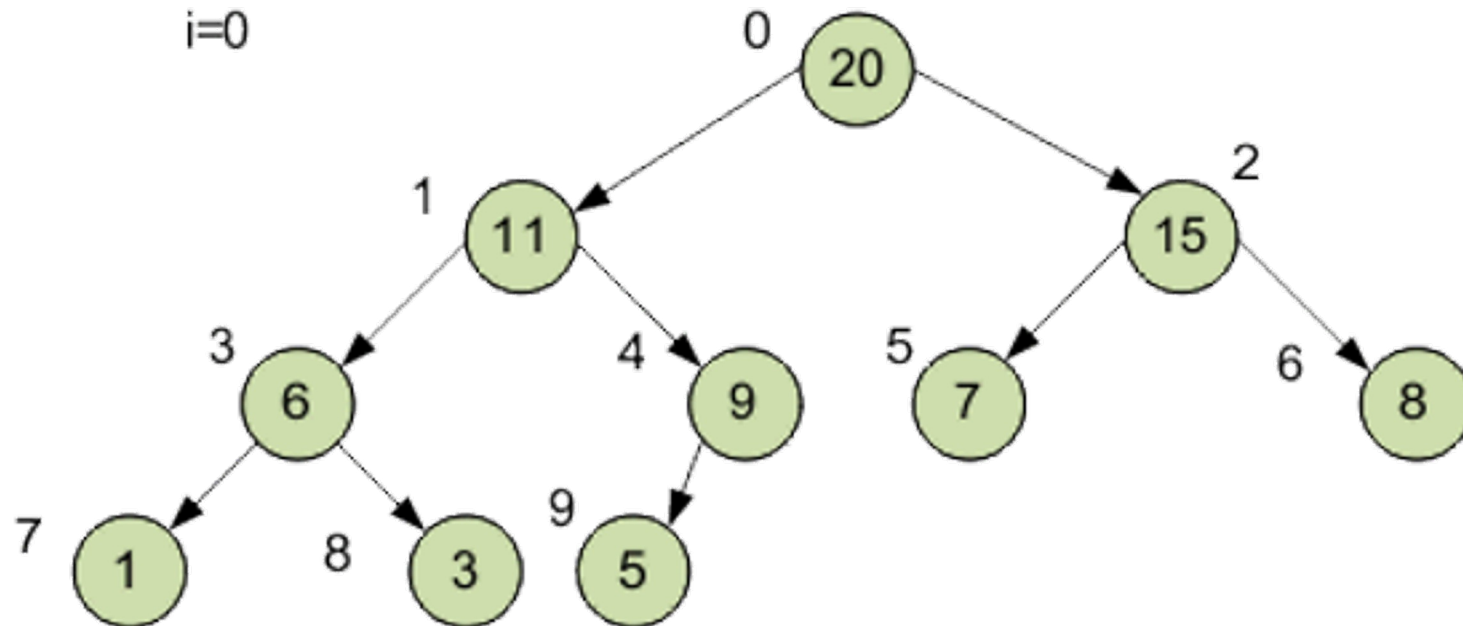
Двоичная куча

- $a[i] \geq a[2i+1]$;
- $a[i] \geq a[2i+2]$.

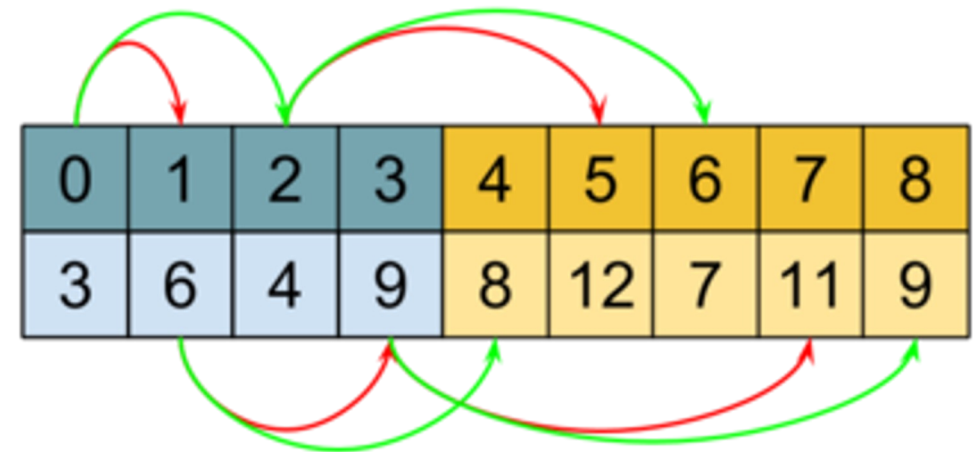
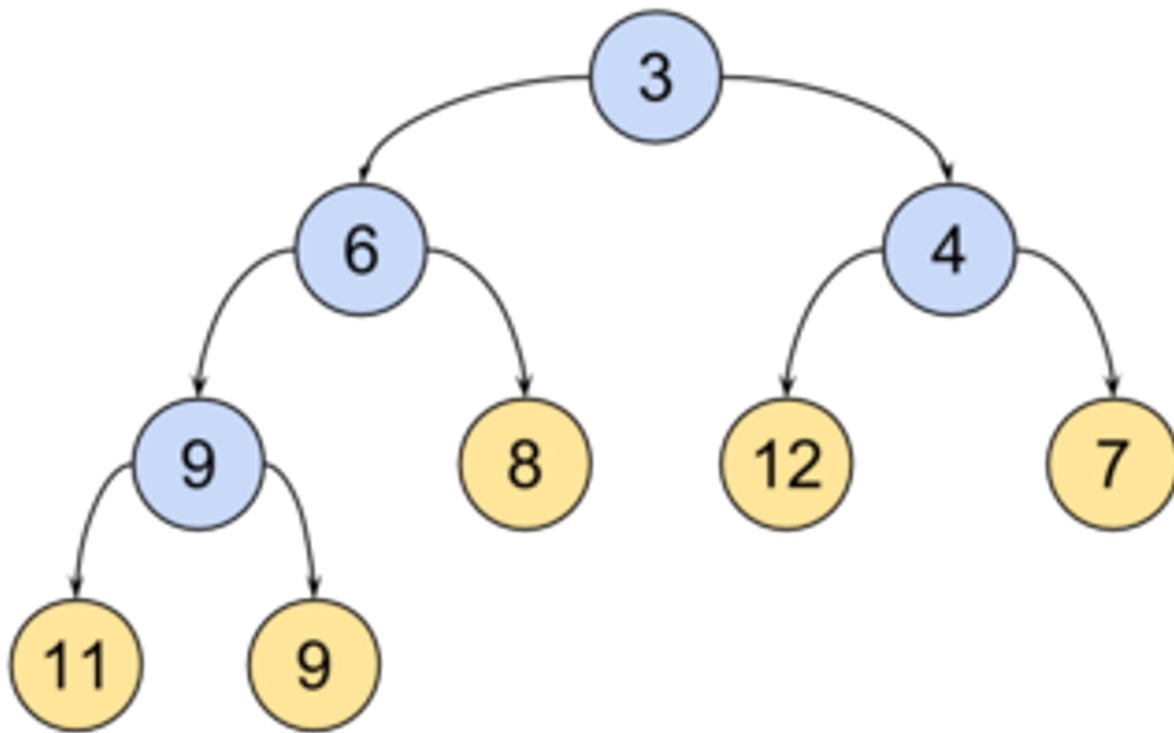


Двоичная куча

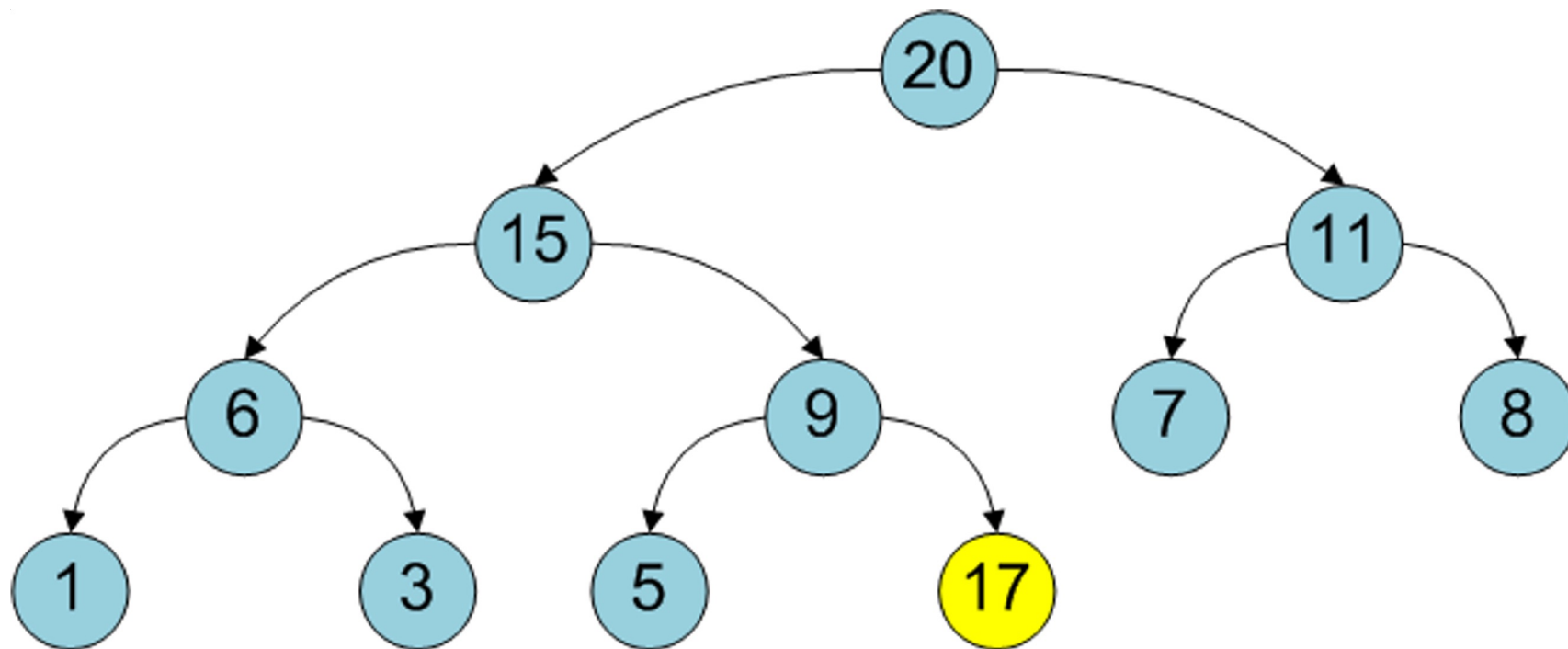
- левый потомок вершины с индексом i имеет индекс $2*i+1$,
- правый потомок вершины с индексом i имеет индекс $2*i+2$,
- Высота двоичной кучи равна высоте дерева, то есть $\log_2 (N+1) \uparrow$,



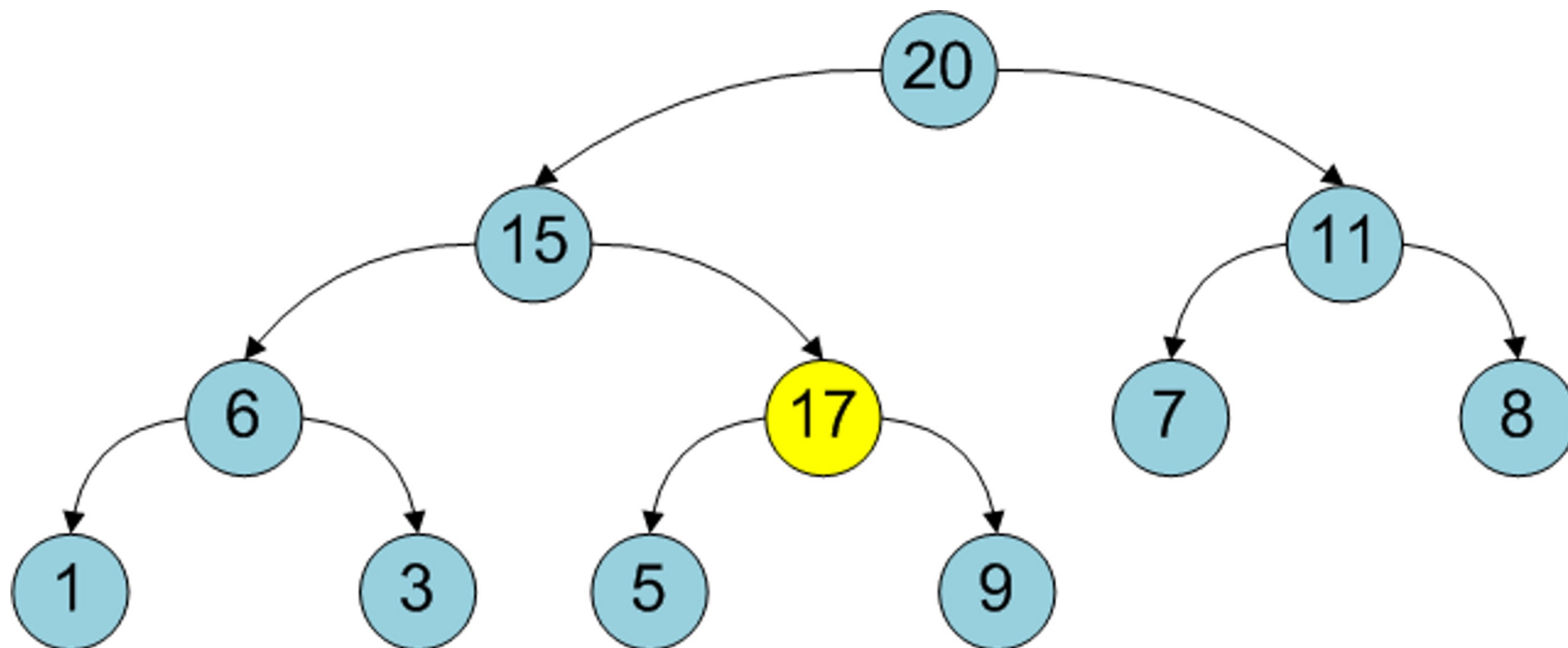
Хранение кучи в массиве



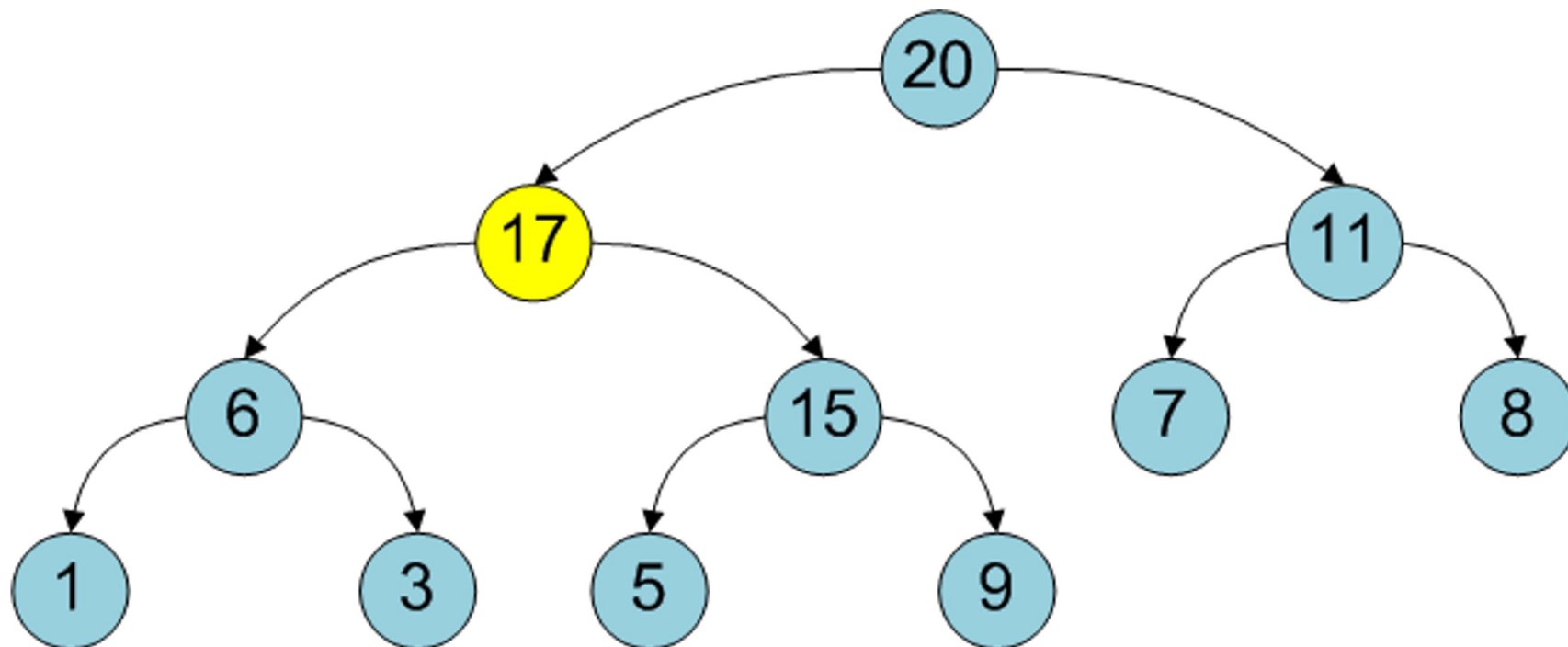
Добавление элемента



Добавление элемента



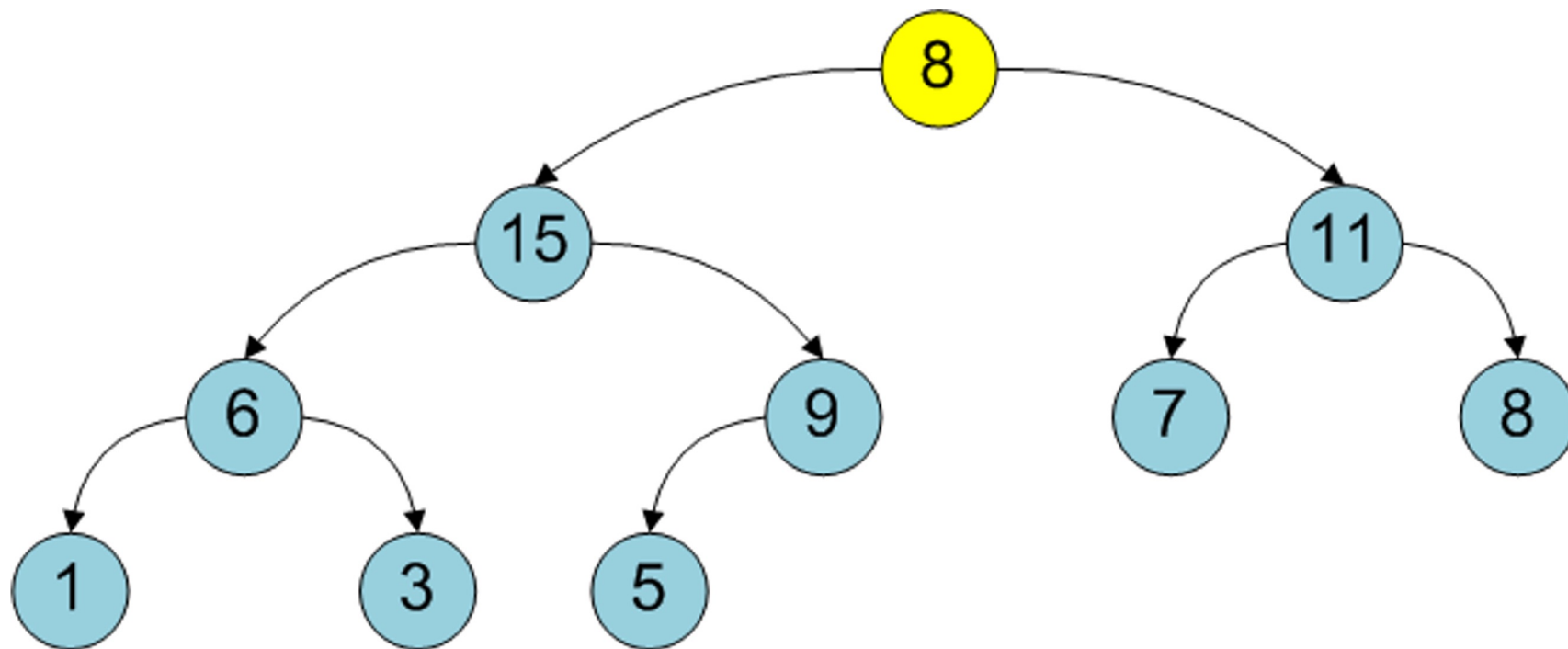
Добавление элемента



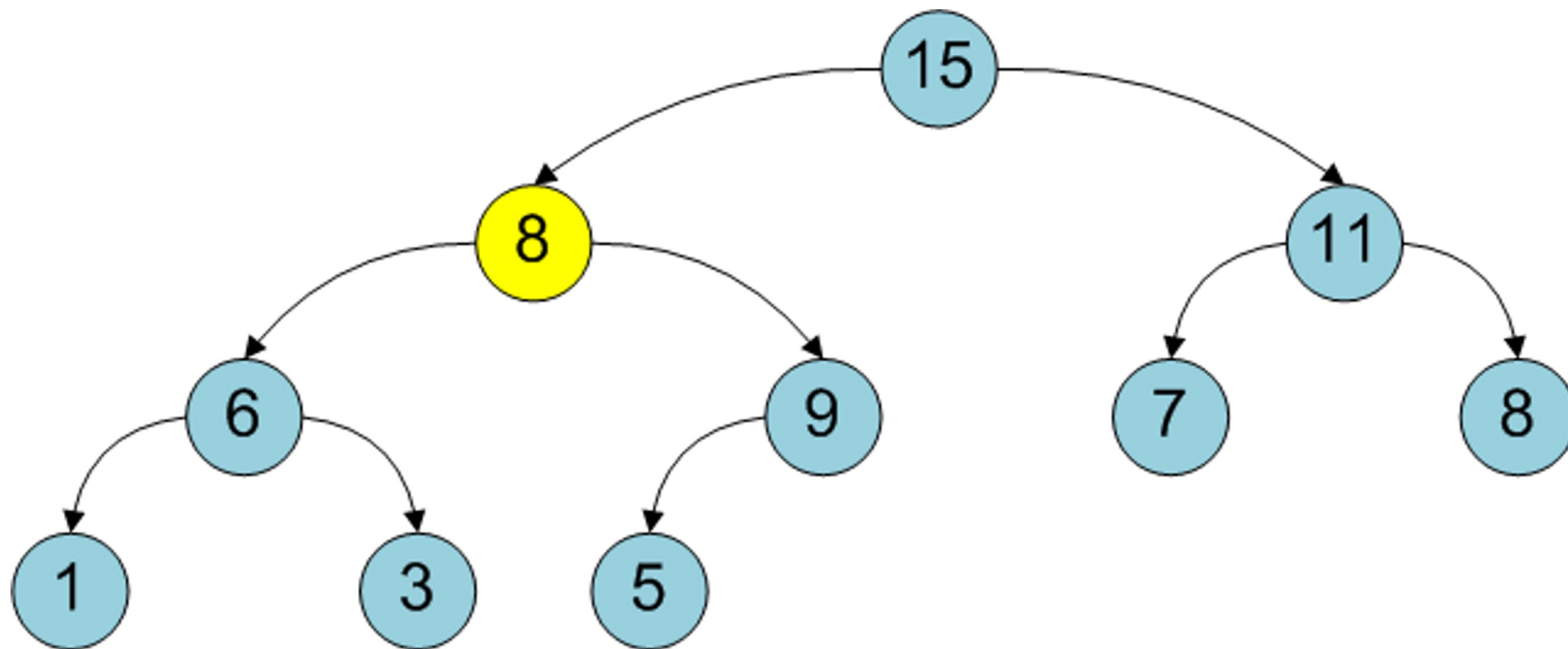
Добавление элемента

```
void add(int value) {  
    list.push_back(value);  
    int i = getHeapSize()  
        - 1;  
    int parent = (i - 1) /  
        2;  
    while (i > 0 &&  
list[parent] < list[i]) {  
        int temp =  
list[i];  
        list[i] =  
list[parent];  
        list[parent] =  
temp;  
        i = parent;  
        parent = (i - 1) /
```

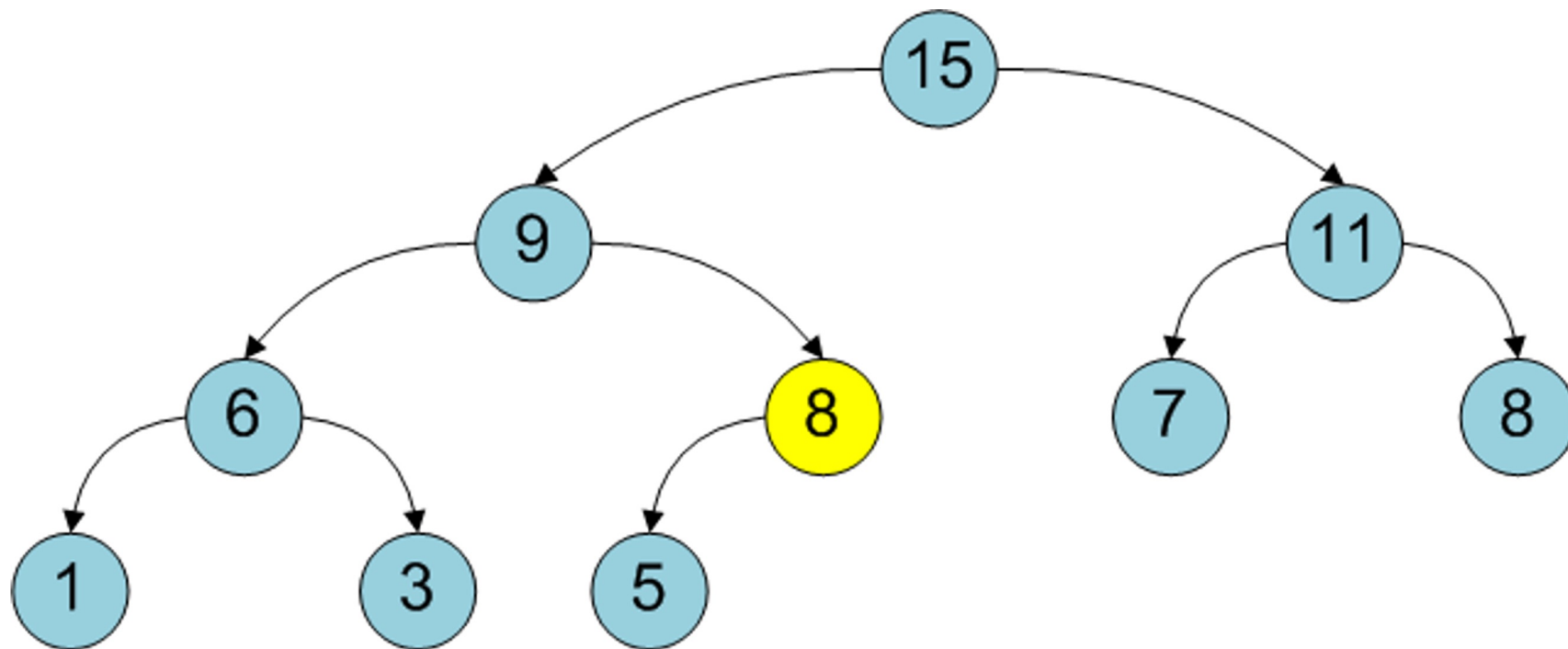
Упорядочение двоичной кучи



Упорядочение двоичной кучи



Упорядочение двоичной кучи



Рекурсивное упорядочивание кучи

```
void heapifyRecursive(int i) {  
    int left_child = 2 * i + 1;  
    int right_child = 2 * i + 2;  
    int largest = i;  
    if (left_child <= getHeapSize() &&  
list[left_child] > list[i]) {  
        largest = left_child;  
    } else {  
        largest = i;  
    }  
    if (right_child <= getHeapSize() &&  
list[right_child] > list[largest]) {  
        largest = right_child;  
    }  
    if (largest != i) {  
        swap(list[i], list[largest]);  
        heapifyRecursive(largest);  
    }  
}
```

Итеративное упорядочивание кучи

```
void heapify(int i) {
    int left_child;
    int right_child;
    int largest_child;
    for (;;) {
        left_child = 2 * i + 1;
        right_child = 2 * i + 2;
        largest_child = i;
        if (left_child < getHeapSize() &&
list[left_child]) {
            largest_child = left_child;
        }
        if (right_child < getHeapSize() &&
list[right_child] > list[largest_child]) {
            largest_child = right_child;
        }
        if (largest_child == i) {
            break;
        }
        int temp = list[i];
        list[i] = list[largest_child];
        list[largest_child] = temp;
        i = largest_child;
    }
}
```

Построение двоичной кучи

```
void buildHeap(vector<int> source_array)
{
    list = source_array;
    for (int i = getHeapSize() / 2; i >=
0; --i) {
        heapify(i);
    }
}
```

Извлечение (удаление) максимального элемента

```
int getMax() {  
    int result =  
list[0];  
    list[0] =  
list[getHeapSize() -  
1];  
  
list.erase(list.end() -  
1);  
    return result;  
}
```

Сортировка с помощью двоичной кучи

Предложена Дж. Уильямсом в 1964 году.

*Williams, J. W. J. (1964), "Algorithm 232 - Heapsort",
Communications of the ACM 7 (6): 347–348.*

- Пирамидальная сортировка (англ. Heapsort, «Сортировка кучей») — алгоритм сортировки, работающий в худшем, в среднем и в лучшем случае (то есть гарантированно) за $O(n \log n)$ операций при сортировке n элементов.
- Количество применяемой дополнительной памяти не зависит от размера массива (то есть, $O(1)$).
- Может рассматриваться как усовершенствованная сортировка пузырьком.

Пирамидальная сортировка

- **1 этап** Построение пирамиды.
- **2 этап** Сортировка на построенной пирамиде. Берем последний элемент массива в качестве текущего. Меняем верхний (наименьший) элемент массива и текущий местами. Текущий элемент (он теперь верхний) просеиваем сквозь $n-1$ элементную пирамиду. Затем берем предпоследний элемент и т.д.

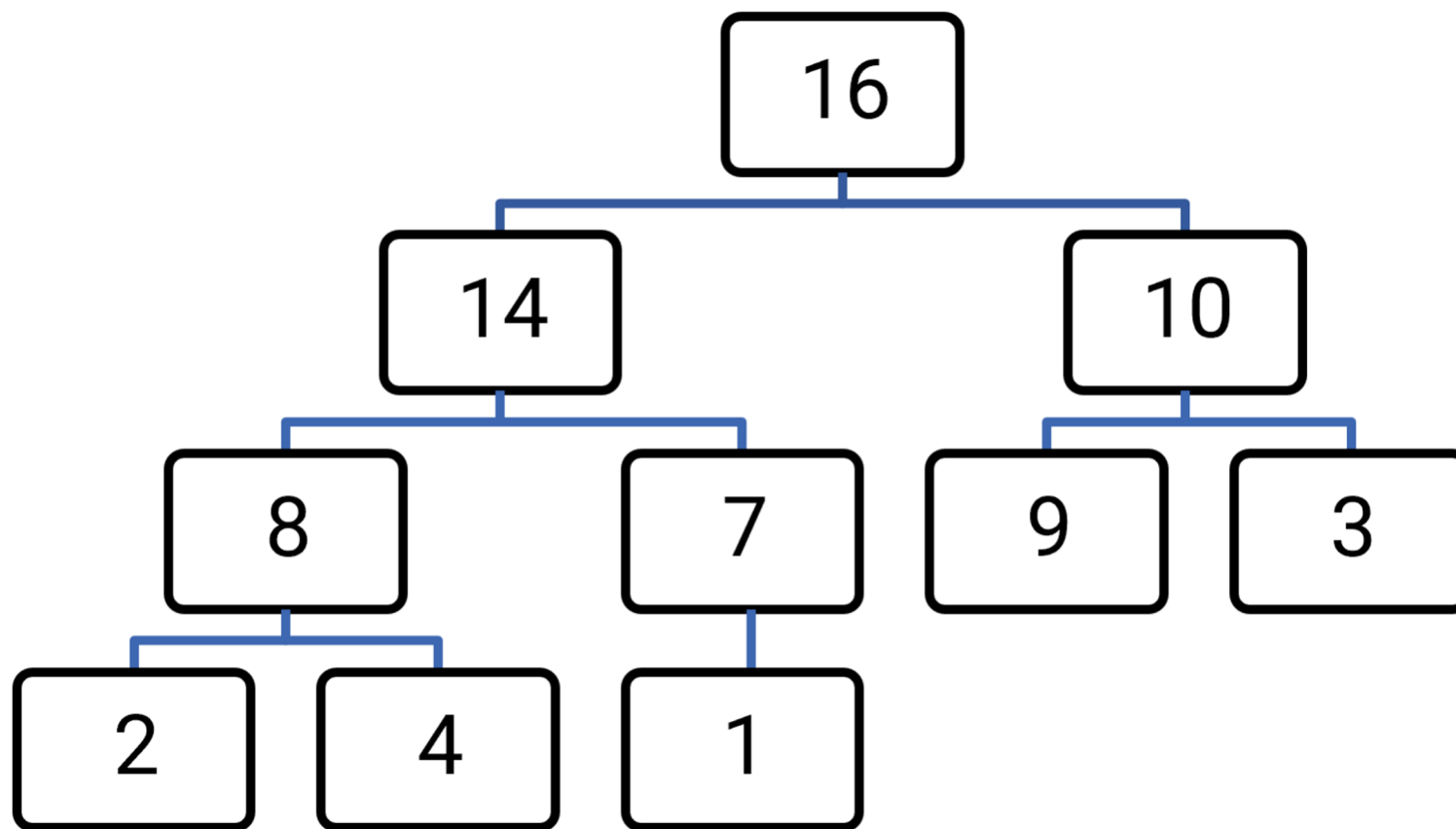
Пирамидальная сортировка. Алгоритм

```
void heapSort(vector<int> array)
{
    buildHeap(array);
    for (int i = array.size() -
1; i >= 0; --i) {
        array[i] = getMax();
        heapify(0);
    }
}
```

Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

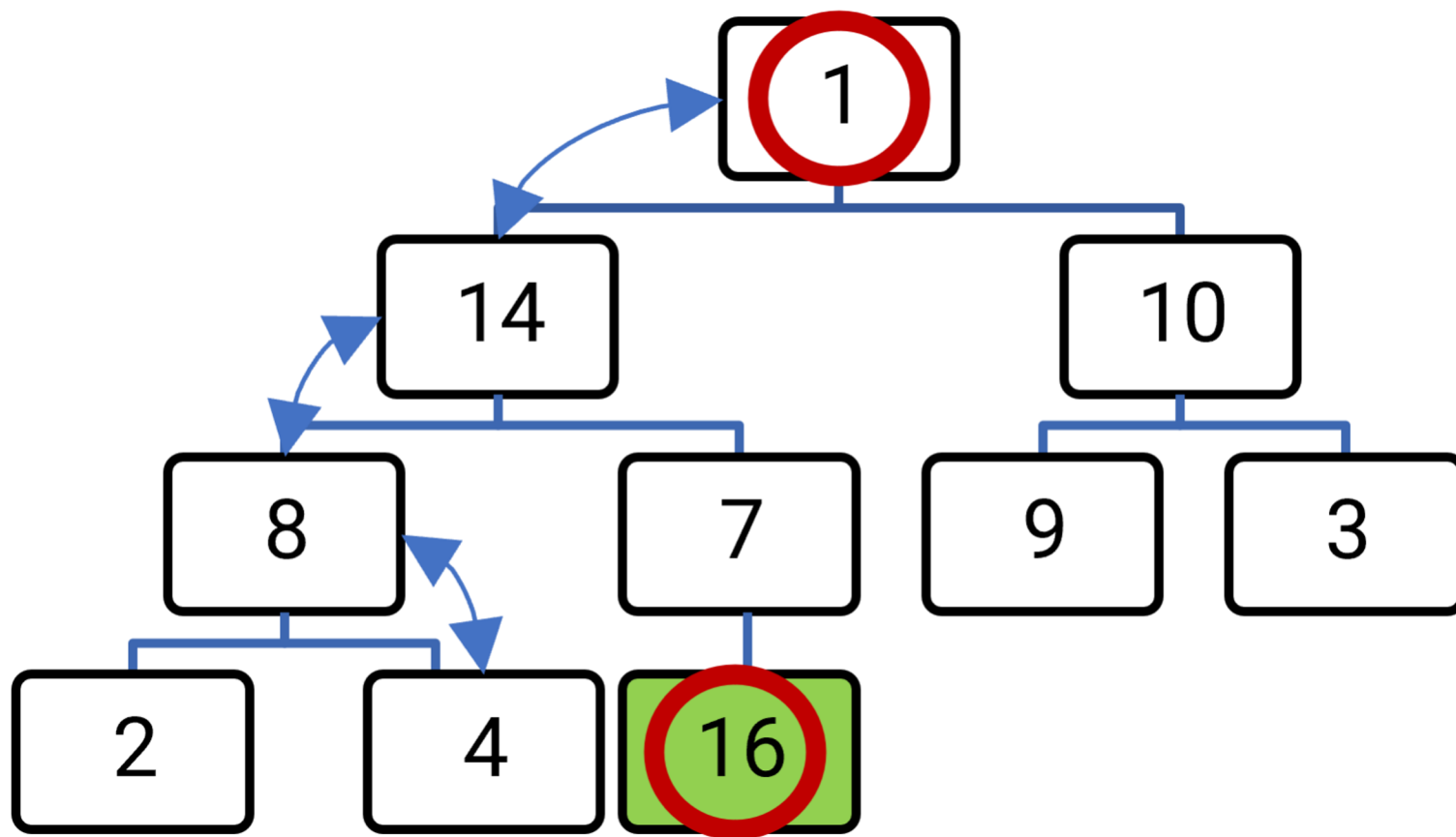
BuildHeap(A)



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

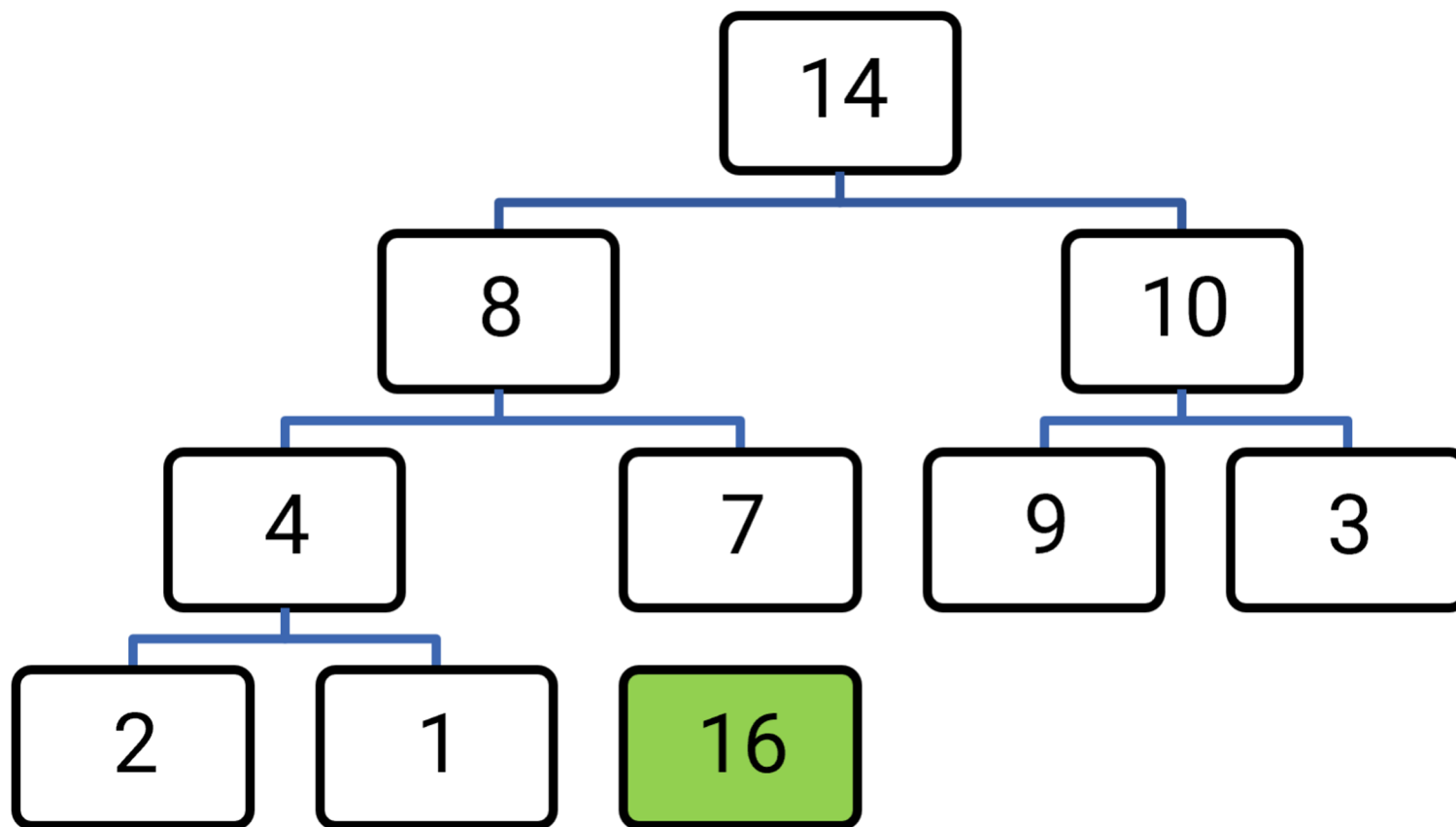
Обмен,
HeapSize-1



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

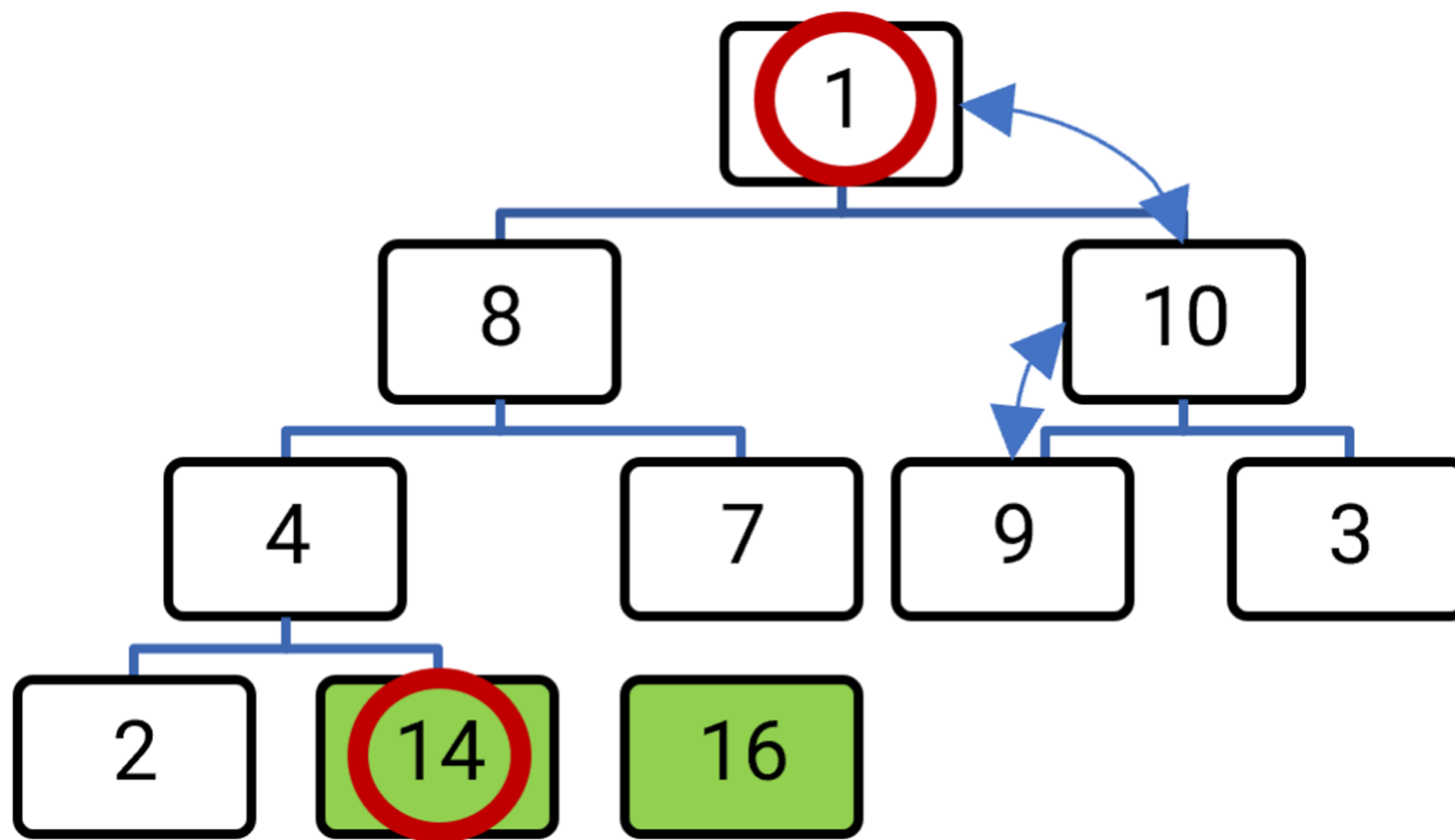
Heapify(0)



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

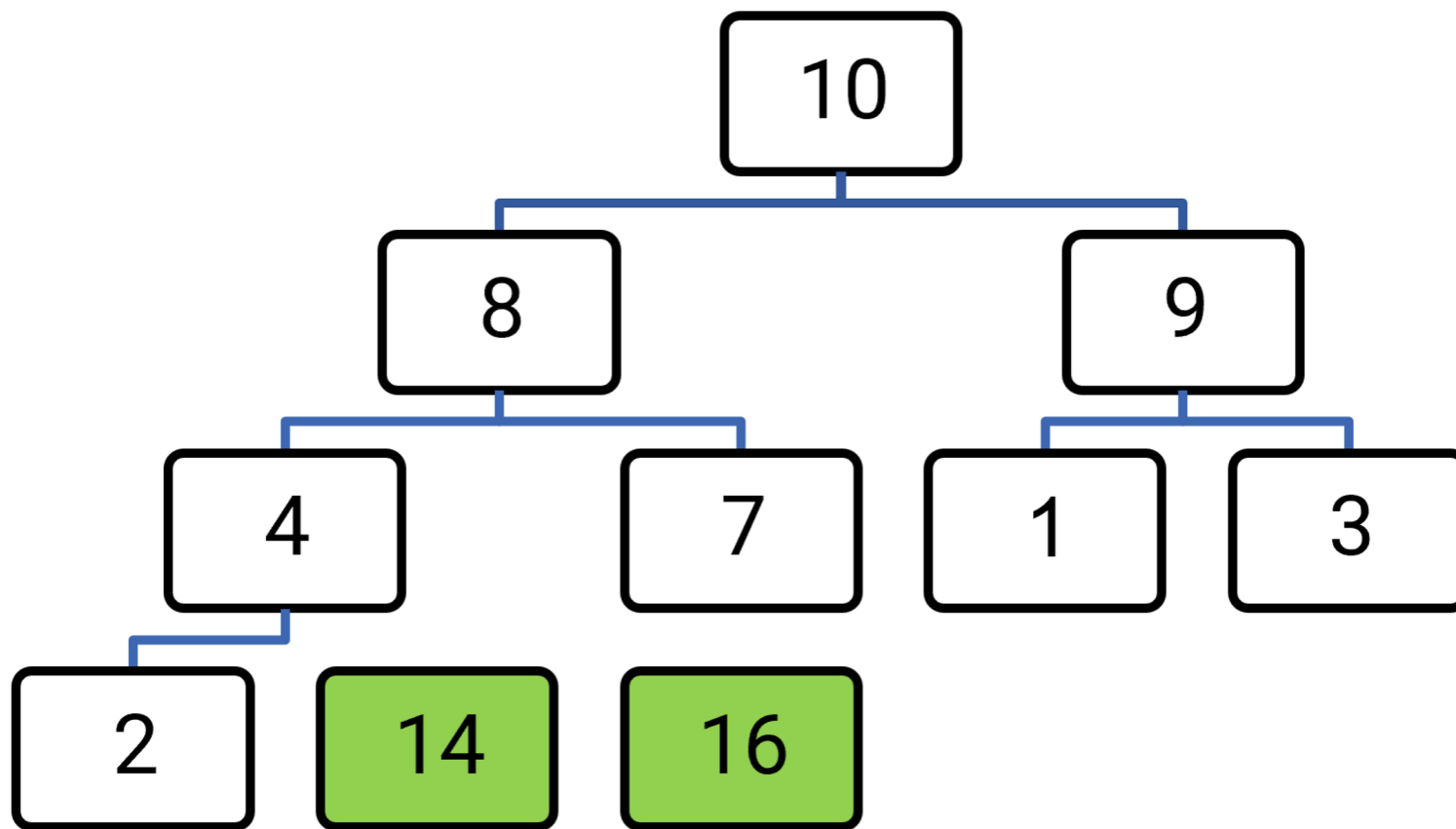
Обмен,
HeapSize-1



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

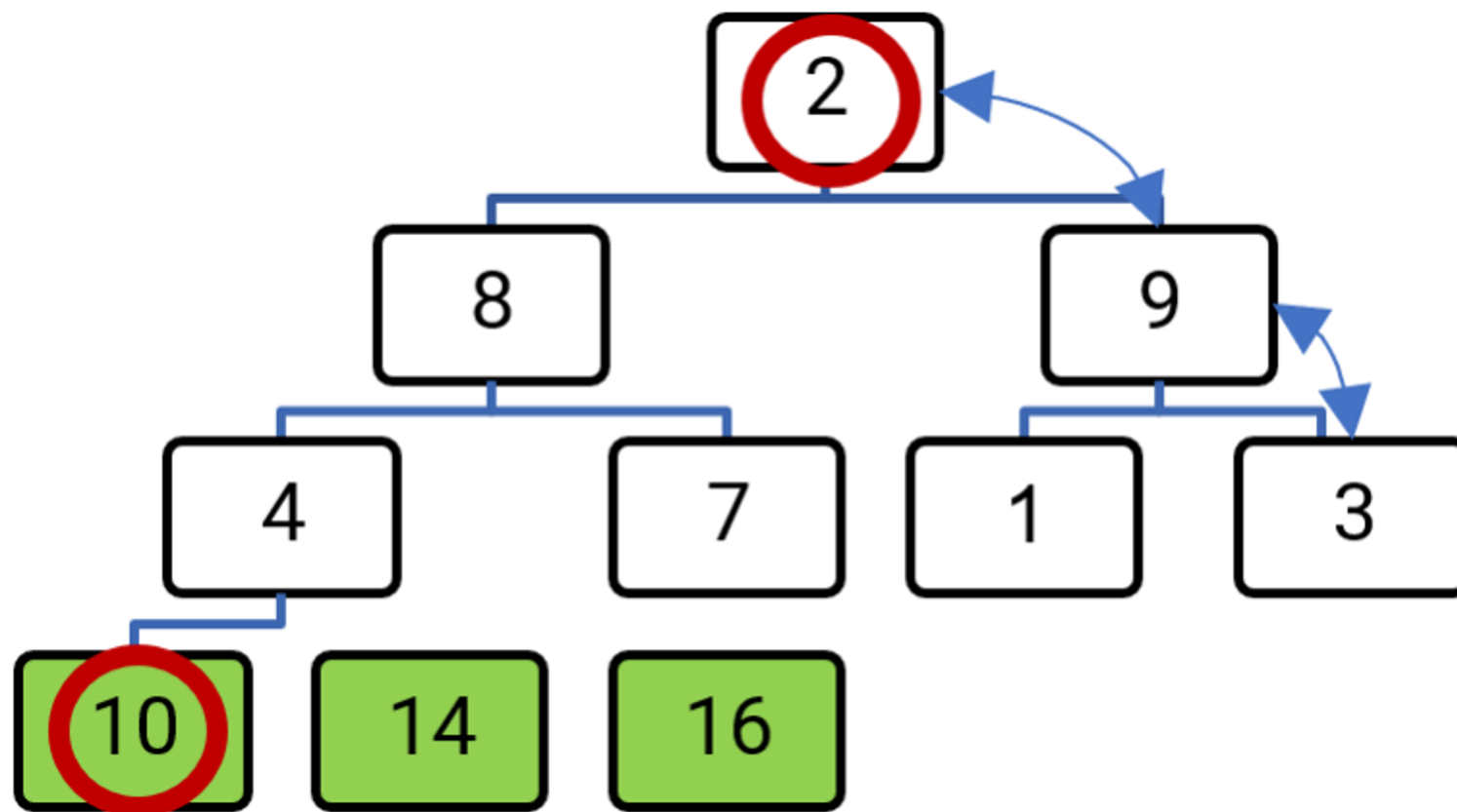
Heapify(0)



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

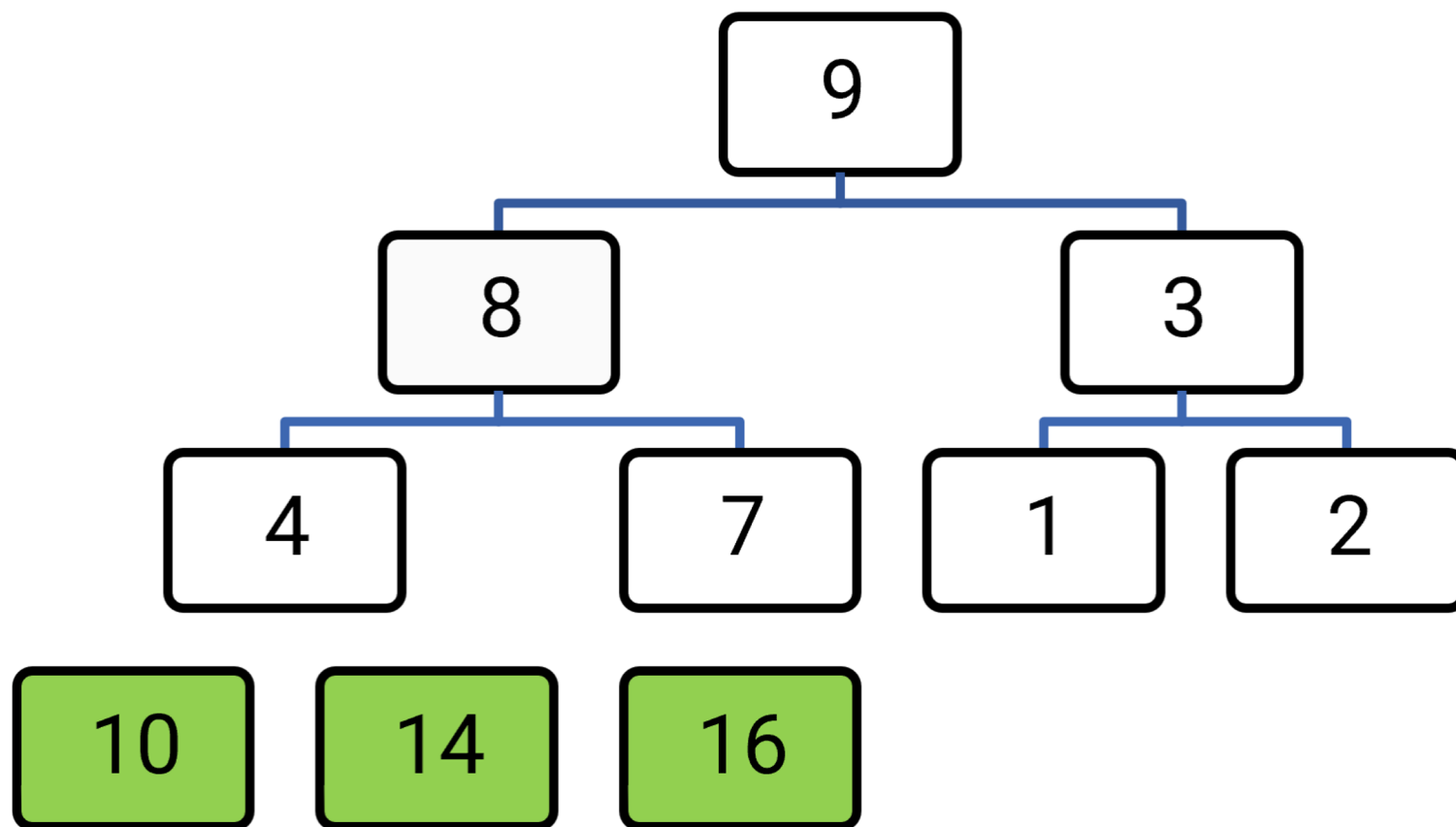
Обмен,
HeapSize-1



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

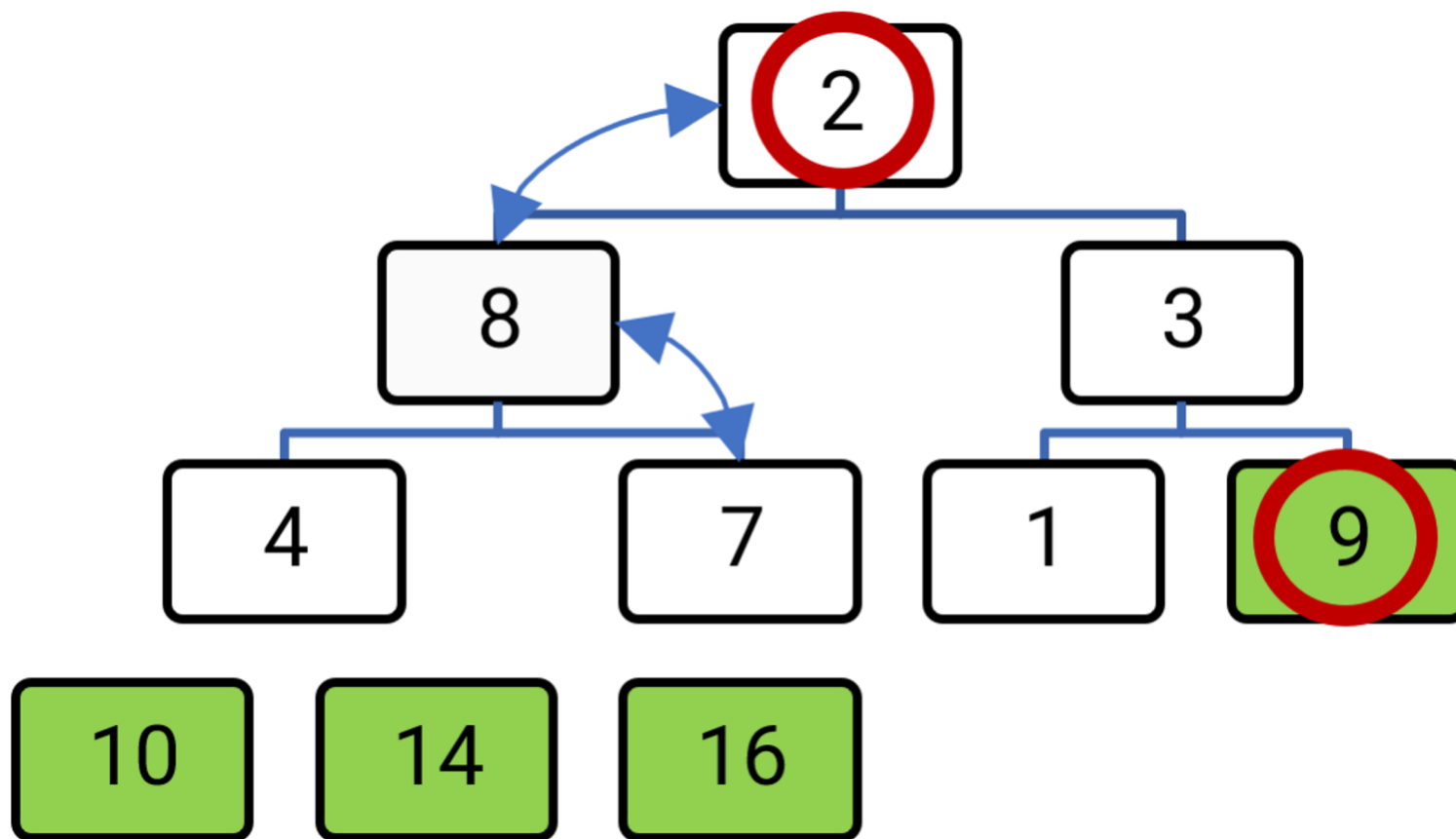
Heapify(0)



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

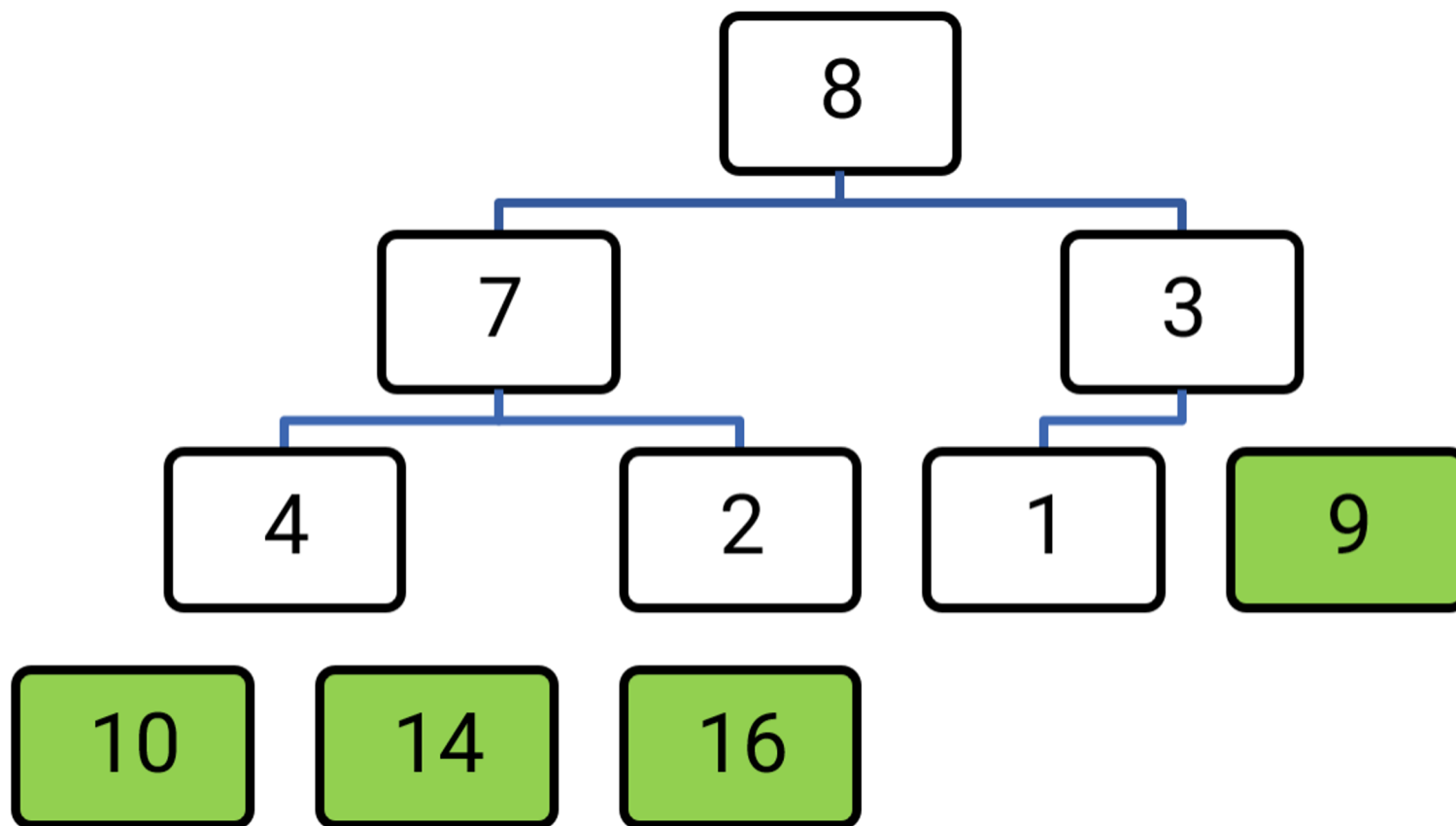
Обмен,
HeapSize-1



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

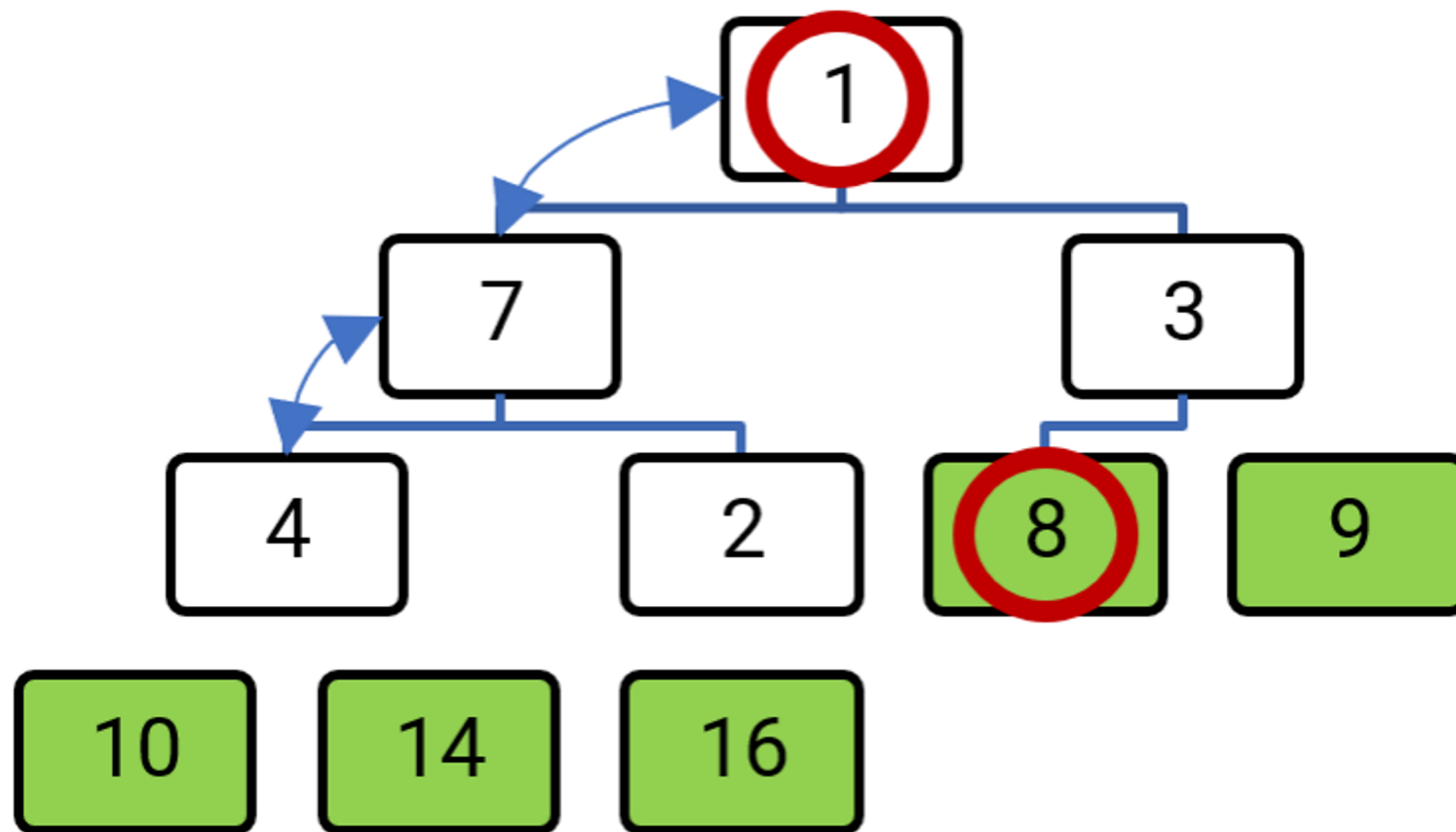
Heapify(0)



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

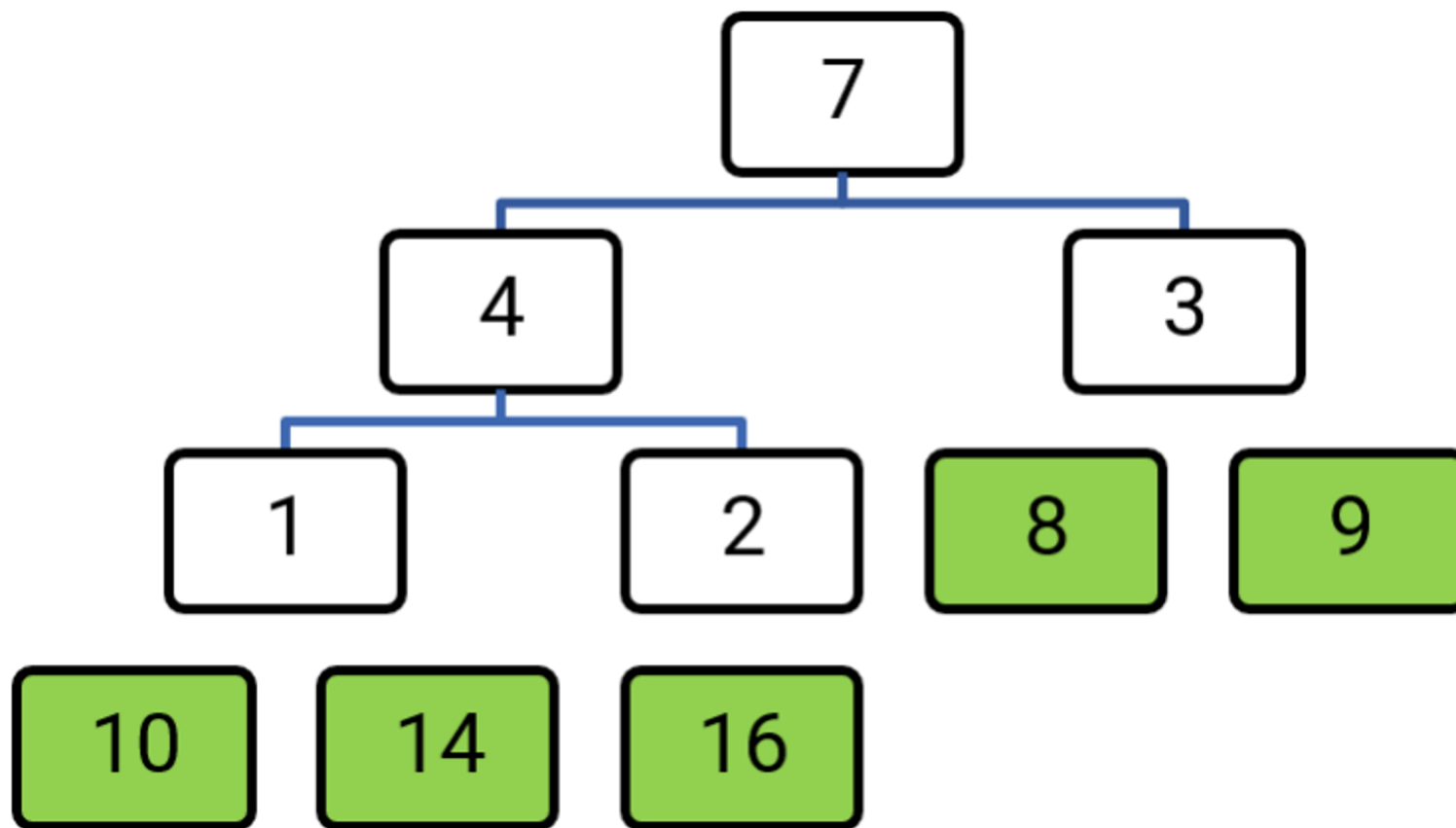
Обмен,
HeapSize-1



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

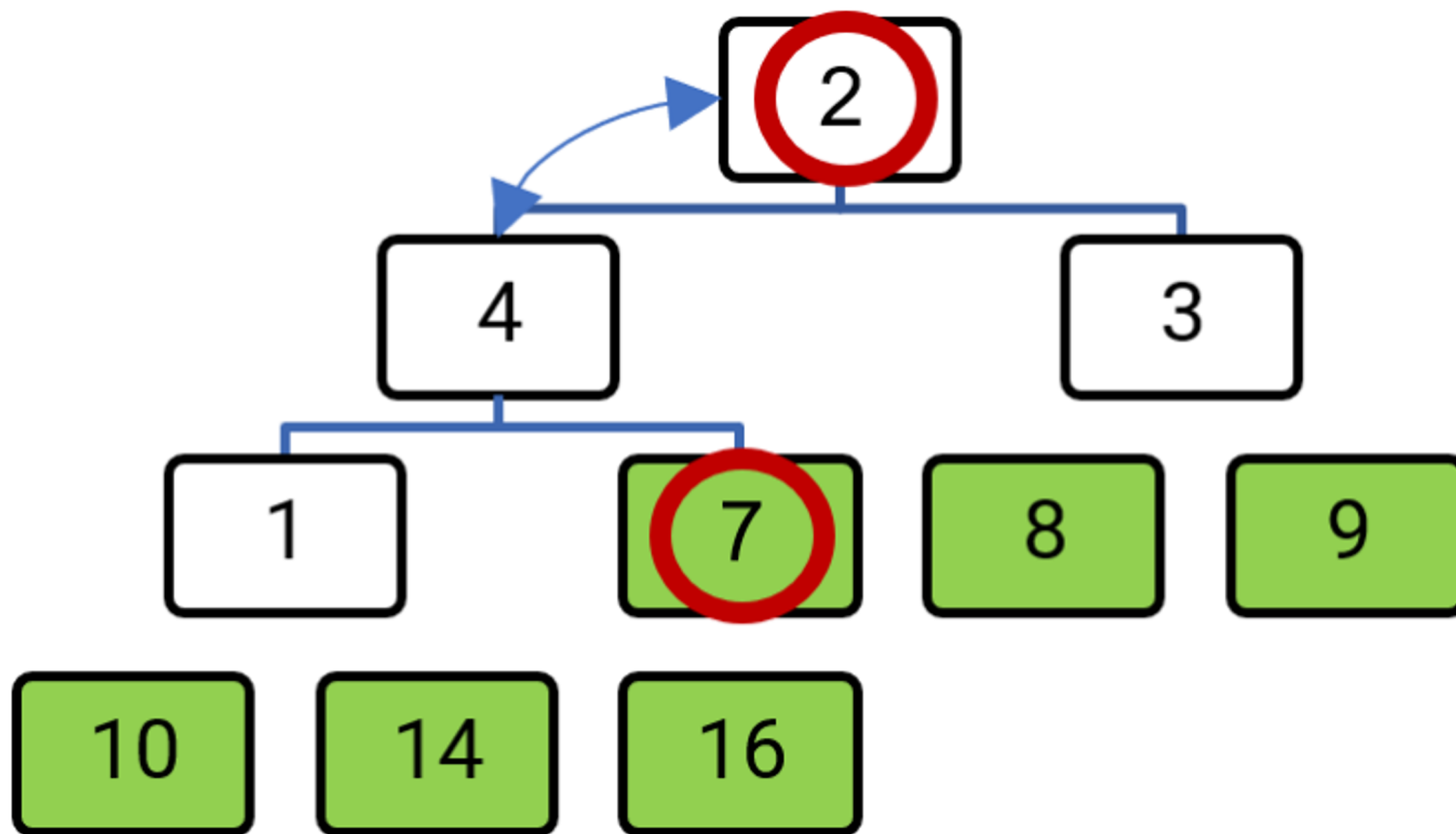
Heapify(0)



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

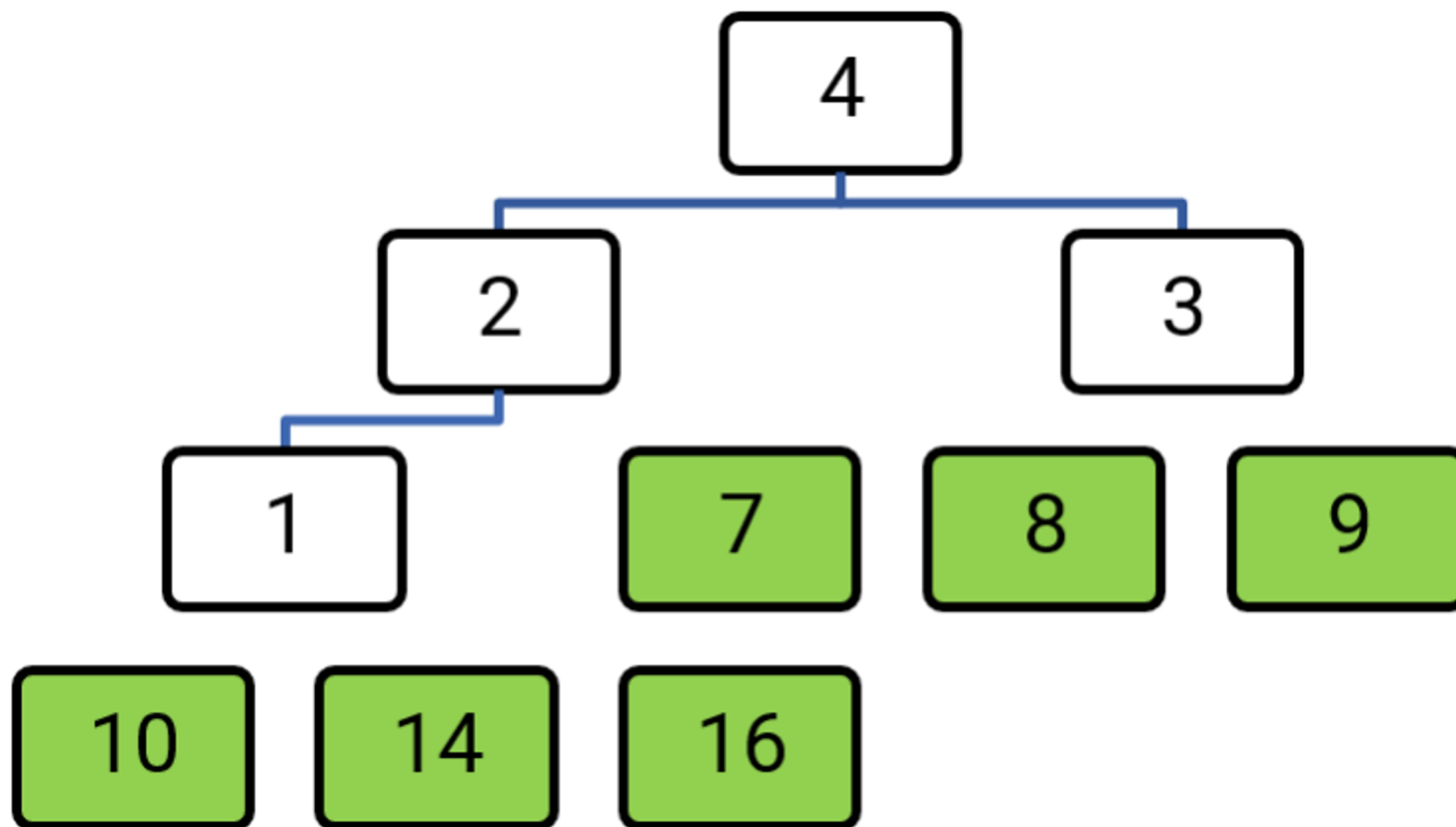
Обмен,
HeapSize-1



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

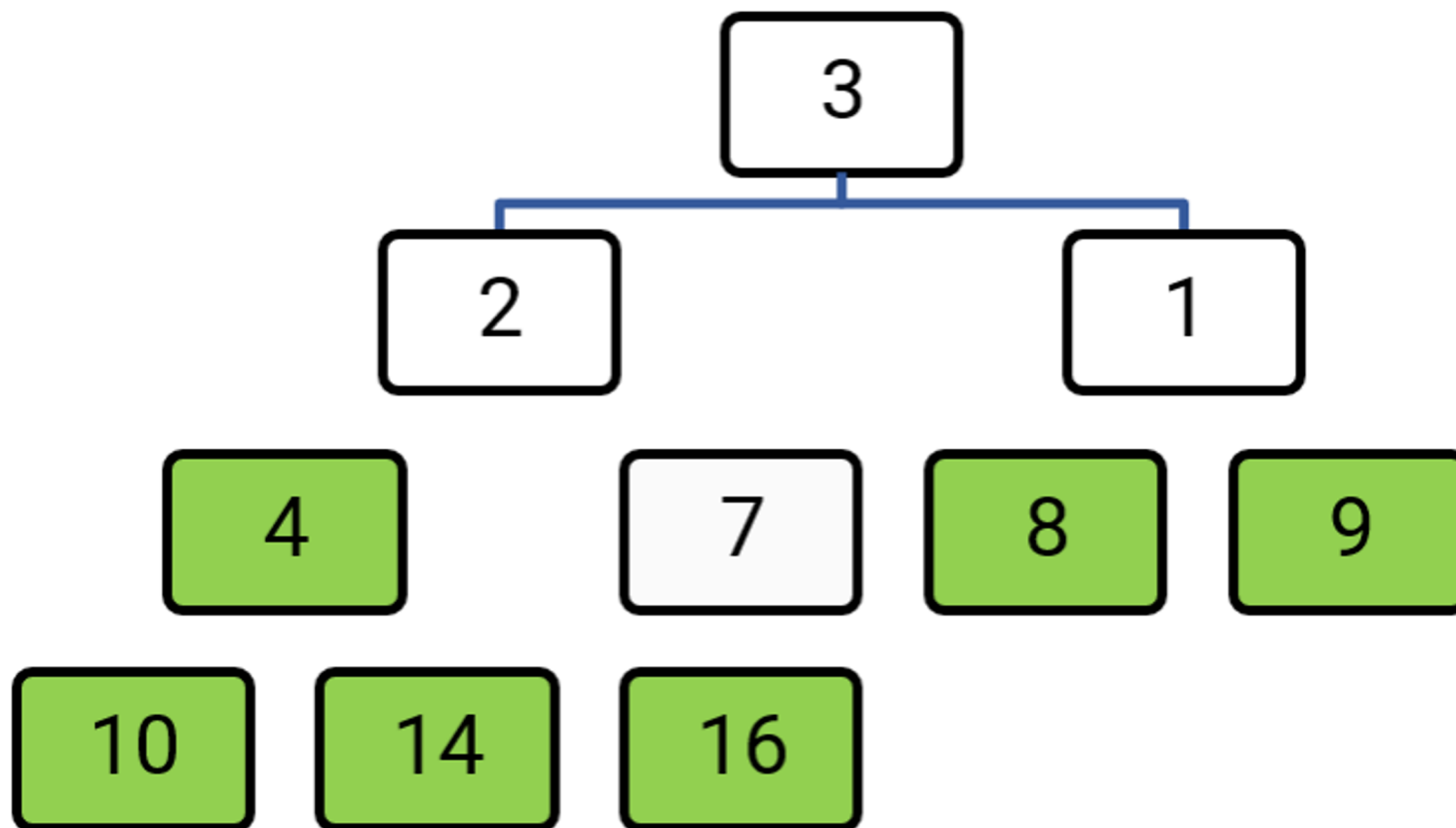
Heapify(0)



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

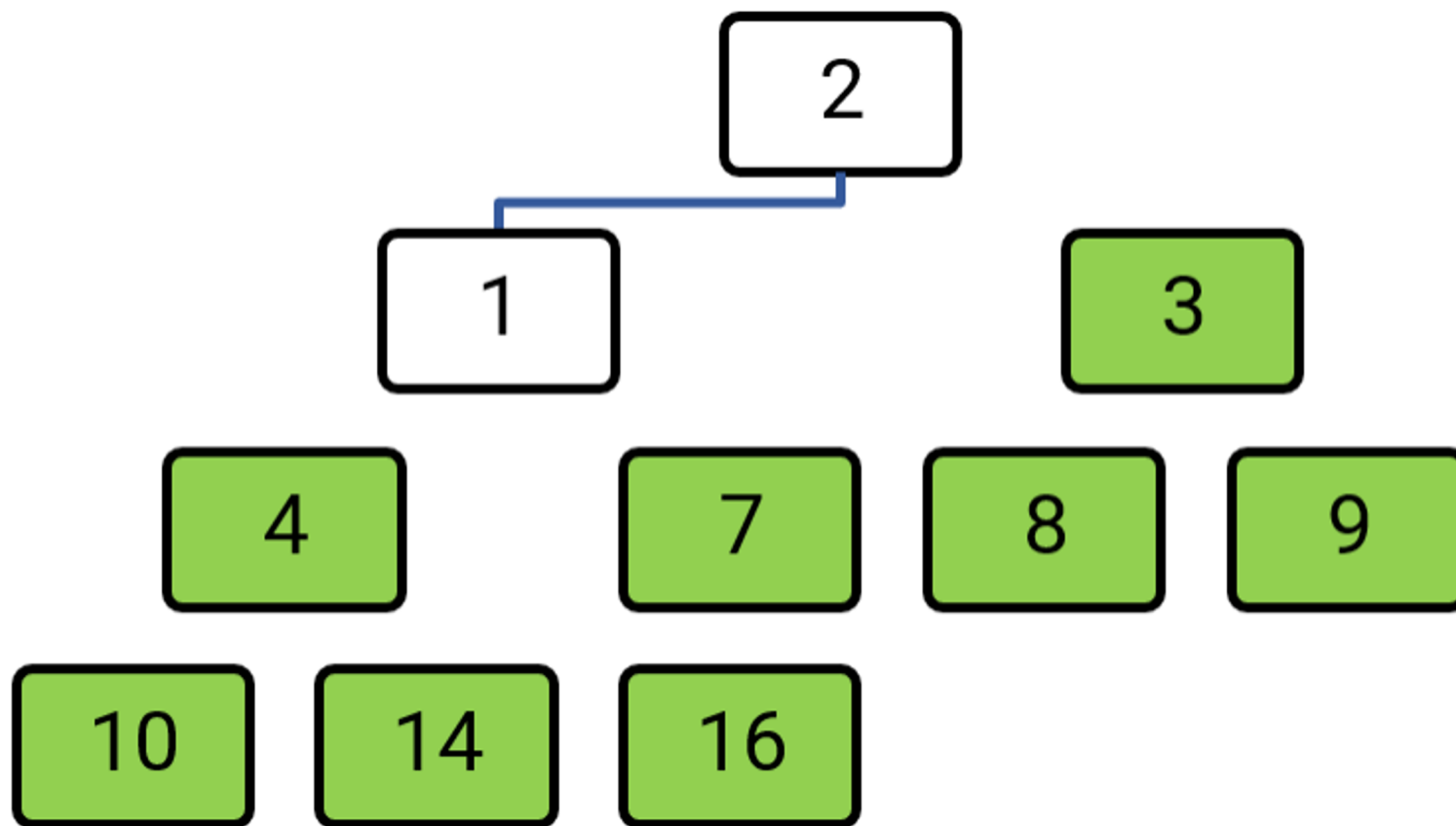
Heapify(0)



Пример:

1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

Heapify(0)



Достоинства и недостатки

- Имеет доказанную оценку худшего случая $O(n \log n)$.
- Сортирует на месте, то есть требует всего $O(1)$ дополнительной памяти
- Неустойчив
- На почти отсортированных массивах работает столь же долго, как и на хаотических данных.
- На одном шаге выборку приходится делать хаотично по всей длине массива — поэтому алгоритм плохо сочетается с кэшированием и подкачкой памяти.
- Методу требуется «мгновенный» прямой доступ; не работает на связанных списках и других структурах памяти последовательного доступа.
- Не распараллеливается.